# Brief Announcement: Towards a Theory of Wear Leveling in Persistent Data Structures

Xialin Liu
University of Waterloo
Canada
x596liu@uwaterloo.ca

Wojciech Golab
University of Waterloo
Canada
wgolab@uwaterloo.ca

## ABSTRACT

The last decade has witnessed an explosion of research on persistent memory, covering both hardware implementations and software techniques. Research activities in this area are primarily driven by the performance benefits of persistent memory, which behaves like DRAM with respect to latency and yet provides the durability of secondary storage. These benefits can only be realized with efficient solutions to the problem of memory cell wear-out, which is one of the fundamental weaknesses of persistent memory versus DRAM, and has traditionally been addressed in hardware. In this paper, we consider the theoretical foundations of solving this problem in software, which allows for application-specific optimizations. Our main contributions are to formalize the problem, and present a novel software implementation of atomic Fetch-And-Increment (FAI) that internally uses multiple words of persistent memory to distribute wear.

## CCS CONCEPTS

• **Theory of computation → Shared memory algorithms**.

## KEYWORDS

persistent memory; wear leveling; concurrency; fault tolerance; theory

**ACM Reference Format:**
Xialin Liu and Wojciech Golab. 2022. Brief Announcement: Towards a Theory of Wear Leveling in Persistent Data Structures. In *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing (PODC '22), July 25–29, 2022, Salerno, Italy*. ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3519270.3538457

## 1 INTRODUCTION

The last decade has witnessed an explosion of research on persistent memory, covering both hardware implementations and software techniques. Research activities in this area are primarily driven by the performance benefits of persistent memory, which behaves like DRAM with respect to latency and yet provides the durability of secondary storage. These benefits can only be realized with efficient solutions to the problem of memory cell wear-out, which is one of the fundamental weaknesses of persistent memory versus DRAM, and has traditionally been addressed in hardware (or firmware). Notably, Intel's recently released Optane Data Center Persistent Memory Module (DCPMM) implements a proprietary wear leveling mechanism that ensures the memory can be overwritten continuously for several years [2].

Intel's proprietary wear leveling technique is a closely guarded secret, but it stands to reason that in the interest of performance, it tracks wear meta-data over blocks of memory cells rather than individual memory cells, similarly to a solid state drive (SSD). For example, empirical studies [3, 7] suggest that the block size may be around 256B or larger for Intel's Optane persistent memory, since accessing Optane modules at such a granularity is known to improve random access bandwidth.

Our key observation in this research is that block-based wear leveling schemes in general allow memory cells within a block to wear unevenly, and so the endurance of the entire block can be ultimately limited by the endurance of a minority of memory cells. For example, a pathological workload could simply cycle a single bit of data on and off, wearing out one cell while not touching the others. The particular data access pattern is less relevant to a flash-based SSD because a block must be erased entirely before any part of it can be overwritten, but becomes relevant for byte-addressable persistent memory since it allows efficient "write-in-place" updates.

In this paper, we consider the theoretical foundations of solving the wear leveling problem for persistent memory in software, which enables workload-aware techniques that compensate for skewed memory access patterns.[1] Specifically, we ask the following abstract research question: if each memory word can only accommodate $T$ updates, then how can one design synchronization objects that can support more than $T$ updates over their lifespan without relying on the persistent memory hardware to identify and remap worn out blocks? Our main contributions in that regard are to formalize the problem, and present a novel software implementation of a strictly linearizable [1] counter that internally harnesses together multiple words of persistent memory to distribute wear. As we show analytically, the counter has asymptotically optimal resilience against wear-out under certain modelling assumptions, and provides greater resilience than a hardware FAI instruction applied directly against a single word of persistent memory.

## 2 THE WEAR LEVELING PROBLEM

For the purposes of this paper, wear leveling is the abstract problem of implementing a concurrent object that tolerates many state changes, while maintaining correctness, from base objects that may

[1] We ignore the interaction between hardware and software implementations of wear leveling in this preliminary research, and leave hardware-software co-design to future work.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
*PODC'22, July 25–29, 2022, Salerno, Italy*
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9262-4/22/07.
https://doi.org/10.1145/3519270.3538457

lose their correctness after relatively few state changes. We formalize an object's ability to tolerate state changes as the following *endurance* property, where $T$ can denote a constant or a function of the number of threads:

*Definition 2.1.* An object has *endurance* $T$ if it maintains its safety and liveness properties in all executions where at most $T$ updates (i.e., operations other than reads) are invoked on the object, but not in some execution where $T + 1$ updates are invoked.

In general, the endurance of an implemented object (e.g., one that is strictly linearizable [1] and lock-free) is limited by the endurance of the base objects (e.g., atomic ones) from which the implemented object is constructed. Several variations on the theme can be considered depending on how the endurance of the base objects is decided. In the simplest case, one can assume for simplicity of analysis that all base objects have the same endurance $T$, and that $T$ is a constant known to the implementation's algorithm. However, the threshold $T$ is difficult to model faithfully as memory cells may wear out non-deterministically in practice, and moreover their actual behaviour may deviate from the manufacturer's informal guarantees. This motivates *endurance-aware* implementations that assume a conservative lower bound on the endurance of the base objects, and *endurance-oblivious* implementations that treat the endurance of the base objects as an unknown. Whereas an endurance-aware implementation could use one base object repeatedly until this object reaches its endurance threshold, and then move on to another base object, an endurance-oblivious implementation might spread wear across multiple base objects more incrementally to maximize the endurance of the implemented object.

## 3 THE COUNTER ALGORITHM

This section presents an endurance-oblivious implementation of an atomic counter from an array of $k$ base objects that support reads and atomic `CompareAndSwap` operations. The $k$ elements are incremented in round-robin fashion. Letting $\langle i_0, i_1, \ldots, i_{k-1} \rangle$ represent the array elements, the counter satisfies the following representation invariant:

(1) The initial state of the counter is $i_j = 0$ for all $0 \leq j < k$.
(2) For all $x, y$ such that $0 \leq x < y < k$, $i_y \leq i_x \leq i_y + 1$ holds.

Intuitively, the invariant is maintained by initializing the array to zeroes, and then incrementing the element $i_{j \bmod k}$ during the $j$'th Fetch-And-Increment (FAI) operation on the implemented counter. For example, if $k = 4$ then the implemented counter object stores the value 3 when the state of the array is $\langle 1, 1, 1, 0 \rangle$, 4 when the state is $\langle 1, 1, 1, 1 \rangle$, and 5 when the state is $\langle 2, 1, 1, 1 \rangle$.

The main technical challenge in implementing the counter in a thread-safe manner is that threads do not know the linearization order of their FAI operations a priori, and so they cannot easily determine the correct array index $j \bmod k$ to update. We solve this problem by applying a simple linear (modulo $k$) search over the array elements, and using the Compare-And-Swap (CAS) instruction to update each element safely. The pseudo-code for the FAI operation is presented in Figure 1. We assume that the hardware provides $b$-bit words for some $b \geq \lceil \log_2 k \rceil$, and that any overflowed values computed at line 5 ($iter + 1$) or line 7 ($index + iter \times k$) are automatically truncated to $b$ bits. The latter assumption implies that the implemented counter is only a $b$-bit counter despite having the capability to represent $k \times 2^b$ distinct states internally under the constraints imposed by the representation invariant.[2] The algorithm, as presented, assumes that the multiprocessor uses a volatile cache, and for this reason there are `Flush` operations at lines 2, 6, 9, and 13. Each `Flush` operation represents a cache line flush or write-back. Additionally, memory barriers may be required for the algorithm to function correctly on a particular hardware platform, and these are omitted for simplicity.

**Persistent shared variables:**

- $A[0..(k-1)]$: array of $b$-bit words, each element initially 0

**Procedure** Fetch-And-Increment()

```
1  iter ← A[k − 1]
2  Flush(&A[k − 1])
3  index ← 0
4  while true do
5      if CompareAndSwap(&A[index], iter, iter + 1) = true
         then
6          Flush(&A[index])
7          return index + iter × k
8      else
9          Flush(&A[index])
10         index ← (index + 1) mod k
11         if index = 0 then
12             iter ← A[k − 1]
13             Flush(&A[k − 1])
```

**Figure 1: Endurance-oblivious counter implementation.**

## 4 ANALYSIS

We now proceed with an analysis of correctness, which assumes an asynchronous collection of threads, and a system-wide crash failure model. We also assume for simplicity of analysis that the size $b$ of a memory word is sufficiently large relative to the speed of the multiprocessor that the implemented counter does not overflow during any one execution of the FAI operation by any thread; this is required for correct behavior of the CAS instruction at line 5.

Theorem 4.1. *The algorithm in Figure 1 implements a strictly linearizable $b$-bit counter.*

Proof Sketch. The linearization point (LP) of the FAI operation is a successful execution of the CAS instruction at line 5, but an incomplete FAI operation is included in the linearization order *only* if the new value written by the CAS is subsequently flushed to persistent memory. The flush can happen in one of three ways: by the action of the same thread at line 6, by the action of another thread executing the helping mechanism at line 2, 9, or 13, or by a spontaneous flush (e.g., due to a cache eviction). In all three cases, the LP occurs before the operation returns a value or the caller crashes, as

[2] We assume that a FAI operation applied on the implemented object in state $2^b - 1$ returns $2^b - 1$ and transitions the abstract state back to 0.

required for strict linearizability. We construct a strict linearization by arranging FAI operations in order of their LPs, and completing any pending operations with a response equivalent to the execution of line 7 using the values of *index* and *iter* corresponding to the LP. To facilitate the rest of the analysis, we establish a crucial claim:

> The $\ell$'th FAI operation in the candidate linearization is the one that executes the $\ell$'th CAS at line 5 that succeeds and whose updates are subsequently flushed to persistent memory. Moreover, this FAI operation executes its successful CAS with $iter = \left\lfloor \ell/2^k \right\rfloor$ and $index = \ell \bmod 2^k$.

The claim broadly follows from the structure of the algorithm, which ensures that the successful CAS instructions cycle through the array elements $0, 1, \ldots, k-1$ in round-robin order, where the iteration number is increased after each round (i.e., after traversing $k$ elements). To prove the first part of the claim, it suffices to show that (i) the effects of successful CAS instructions in the same round are flushed in the right order, and (ii) the same holds regarding the last successful CAS of one round versus the first successful CAS of next round. Assuming that the implemented counter does not overflow during any one execution of the FAI operation, as noted earlier, point (i) follows from the helping mechanism at line 9 and point (ii) follows from the helping mechanism at line 2 and line 13. The second part of the claim, regarding *iter* and *index*, then follows immediately from the traversal order of the $k$ array elements, which we described at the beginning of this proof.

To complete the proof of Theorem 4.1, it suffices to show that the $\ell$'th operation in the candidate linearization returns $\ell \bmod 2^b$. This follows from the second point of our claim above, our earlier assumption that arithmetic results are truncated to $b$ bits, and from the formula evaluated at line 7 to compute the return value. □

Theorem 4.2. *The algorithm in Figure 1 is lock-free.*

Proof Sketch. Suppose for contradiction that in some execution there is a collection of one or more threads executing the FAI operation concurrently, and these threads take infinitely many steps without crashing or completing any FAI operation. Then the execution has a suffix $S$ in which no successful CAS operation occurs at line 5, which implies that the state of the array $A$ is constant. Eventually, some thread must read a value $v$ from $A[k-1]$ at line 13, and then apply CAS to the same element at line 5 with arguments $v$ and $v+1$, after performing $k$ additional iterations of the loop. Since the state of $A$ is constant in the suffix $S$, the value of $A[k-1]$ remains $v$ immediately before the CAS instruction, which contradicts the earlier point that all CAS instructions fail in the suffix $S$. □

The final piece of the analysis addresses the resilience of the counter implementation to memory cell wear-out. We assume for simplicity of analysis that all base objects have endurance $T$, and derive the endurance of the implemented counter.

Theorem 4.3. *If the elements of the array $A$ individually have endurance at $T$, then the implemented counter object has endurance $k \times T$.*

Proof Sketch. We assume that the multiprocessor provides a volatile write-back cache, which implies that persistent memory is updated by cache line flushes or write-backs, and not directly by the CAS instructions. This can occur at lines 2, 6, 9, or 13 by the action of a `Flush` operation, or by a spontaneous system-initiated cache line eviction. To continue the analysis, we make a series of assumptions regarding the behavior cache line flushes and write-backs:

(1) These operations only affect one element of $A$ at a time, e.g., because the elements of $A$ are cache line aligned.
(2) These operations only update persistent memory if the cache line under consideration is found in a dirty state, for example Modified or Owned in the MOESI coherence protocol, or Modified in the MESIF protocol [6].
(3) These operations never leave a cache line in a dirty state, hence their repeated application on the same address updates the persistent memory at most once.

We also assume that a failed CAS instruction does not place the cache line in a dirty state, and so the persistence operations have no effect when applied immediately after a failed CAS. Under the above assumptions, each element of the array $A$ is updated at most once per successful CAS instruction. Furthermore, each FAI operation on the implemented object updates at most one element of $A$, namely at the point of a successful CAS instruction at line 5. Since the array elements are updated in round-robin order and there are $k$ elements, it follows that $k \times T$ implemented FAI operations can be invoked before any array element is updated more than $T$ times and wears out. Furthermore, when FAI operation $k \times T + 1$ takes effect, element $A[0]$ is accessed $T+1$ times, which exceeds its endurance limit, and this may cause the implemented object to lose correctness. □

Finally, we establish in Theorem 4.4 below that the endurance bound stated in Theorem 4.3 is optimal. For comparison, a hardware FAI instruction followed by a `Flush` provides endurance $T$ while using $k$-fold less space.

Theorem 4.4. *Any implemented object that internally uses $k$ base objects of endurance $T$ can itself guarantee endurance at most $k \times T$.*

Proof Sketch. Under sequential access, each update operation on the implemented object must update at least one base object. Since there are $k$ base objects, at most $k \times T$ such updates can occur before some base object reaches its endurance limit (i.e., wears out), and so the implemented object has endurance at most $k \times T$. □

## 5 CONCLUSION

This paper presented preliminary results on the path to building a theory of wear leveling in persistent data structures. We showed that a software implementation of an atomic counter can provide superior endurance to a hardware FAI instruction in a model where memory wears out according to a deterministic threshold. Our results extend and enhance prior theoretical work on tolerating memory failures through redundancy [4, 5] by presenting a low-level quantitative analysis of the endurance of persistent objects.

## ACKNOWLEDGMENTS

We thank the anonymous reviewers for their insightful feedback. This research was supported by an Ontario Early Researcher Award, a Google Faculty Research Award, as well as the Natural Sciences and Engineering Research Council (NSERC) of Canada.

## REFERENCES


[1] Marcos K. Aguilera and S. Frølund. 2003. *Strict linearizability and the power of aborting.* Technical Report HPL-2003-241. Hewlett-Packard Labs.

[2] Frank Hady. 2019. Intel Optane Technology Delivers New Levels of Endurance. https://www.intel.com/content/www/us/en/architecture-and-technology/optane-technology/delivering-new-levels-of-endurance-article-brief.html

[3] Joseph Izraelevitz, Jian Yang, Lu Zhang, Juno Kim, Xiao Liu, Amirsaman Memaripour, Yun Joon Soh, Zixuan Wang, Yi Xu, Subramanya R. Dulloor, Jishen Zhao, and Steven Swanson. 2019. Basic Performance Measurements of the Intel Optane DC Persistent Memory Module. *CoRR* abs/1903.05714 (2019). arXiv:1903.05714 http://arxiv.org/abs/1903.05714

[4] Prasad Jayanti, Tushar Deepak Chandra, and Sam Toueg. 1998. Fault-Tolerant Wait-Free Shared Objects. *Journal of the ACM* 45, 3 (1998), 451–500.

[5] Thomas Moscibroda and Rotem Oshman. 2011. Resilience of mutual exclusion algorithms to transient memory faults. In *Proc. of the 30th ACM Symposium on Principles of Distributed Computing (PODC).* 69–78.

[6] David A. Patterson and John L. Hennessy. 2012. *Computer Organization and Design - The Hardware / Software Interface (Revised 4th Edition).* Academic Press.

[7] Ivy Bo Peng, Maya B. Gokhale, and Eric W. Green. 2019. System evaluation of the Intel optane byte-addressable NVM. In *Proc. of the International Symposium on Memory Systems (MEMSYS).* 304–315.